

Investigación sobre el desarrollo de aplicaciones móviles

ANDROID

Descripción:

Es un sistema operativo móvil de código abierto basado en linux, diseñado principalmente para dispositivos táctiles como teléfonos inteligentes y tabletas. Fue desarrollado originalmente por Android Inc., empresa que Google compró en 2005 para lanzar el sistema comercialmente en 2008. Su modelo de negocio se basa en la distribución gratuita del software a los fabricantes mediante el consorcio Open Handset Alliance, para que ellos lo adapten. Es el sistema operativo más utilizado del mundo.

Características:

Arquitectura basada en Linux: Gestiona los controladores de hardware, la memoria y la seguridad del sistema.

Entorno de ejecución moderno: Utiliza la máquina virtual ART (Android Runtime) para ejecutar aplicaciones de forma rápida.

Componentes reutilizables: Las aplicaciones pueden compartir funcionalidades y datos entre sí de forma segura.

Sistema de notificaciones: Ofrece un centro unificado y priorizado para las alertas del usuario.

Compatibilidad de hardware: Soporta pantallas plegables, sensores biométricos, acelerómetros, GPS, conectividad 5G, cámaras...

Herramientas de Desarrollo (SDK y Entornos)

Android Studio: El entorno de desarrollo integrado (IDE) oficial basado en IntelliJ IDEA de JetBrains.

Android SDK: Conjunto de herramientas, librerías y emuladores necesarios para compilar y probar las aplicaciones.

Gradle: El sistema automatizado de gestión de proyectos que compila y empaqueta la aplicación en archivos apk

Firebase: Plataforma de Google integrada para añadir bases de datos en tiempo real, analíticas y notificaciones push.

Lenguajes de programación:

Kotlin: El lenguaje oficial y preferido por Google desde 2019. Es moderno, seguro y reduce el código repetitivo.

Java: El lenguaje histórico de Android. La gran mayoría del código antiguo y de las librerías base están escritas en Java.

C++ / C: Utilizados mediante el Kit de Desarrollo Nativo (NDK) para tareas de alto rendimiento como videojuegos o motores gráficos.

Jetpack Compose: La herramienta moderna de Google para diseñar pantallas usando código puramente declarativo.

iOS

Descripción:

es un sistema operativo móvil de código cerrado y propietario desarrollado por Apple Inc. Fue lanzado originalmente en 2007 exclusivamente para el iPhone original (bajo el nombre de iPhone OS). A diferencia de Android, Apple mantiene un control absoluto sobre el hardware y el software: iOS se diseña exclusivamente para los procesadores y componentes de la línea iPhone. Su filosofía se centra en la optimización del rendimiento, la fluidez de las animaciones, la privacidad estricta del usuario y una integración perfecta con el resto del ecosistema de Apple (Mac, iPad, Apple Watch) .

Características:

Núcleo Darwin (Unix): Basado en tecnologías BSD y Mach, lo que le otorga una estabilidad, solidez y seguridad de nivel industrial.

Ejecución Nativa Directa: El código se compila directamente para la arquitectura del procesador (chips de la serie A de Apple), sin necesidad de máquinas virtuales, logrando una eficiencia energética y rendimiento altísimos.

Sandbox de Seguridad: Cada aplicación se ejecuta en su propio entorno aislado ("caja de arena"). Ninguna app puede acceder a los datos de otra ni del sistema sin permisos explícitos.

Ecosistema cerrado: El usuario solo puede instalar aplicaciones revisadas y aprobadas manualmente por Apple a través de su tienda oficial.

Ciclo de Actualizaciones: Apple ofrece soporte directo de actualizaciones de software simultáneas para dispositivos de hasta 5 o 6 años de antigüedad.

Lenguajes de Programación:

Swift: El lenguaje oficial y moderno introducido por Apple en 2014. Es rápido, seguro por diseño, interactivo y cuenta con una sintaxis limpia que reemplazó las complejidades del software antiguo.

Objective-C: El lenguaje histórico de Apple. Basado en C y Smalltalk, fue el pilar sobre el que se construyó todo iOS en sus inicios. Hoy en día se mantiene por compatibilidad en proyectos heredados.

C++ / C: Utilizados en motores de videojuegos o librerías de procesamiento de audio/video de muy bajo nivel.

SwiftUI: framework declarativo moderno de Apple para diseñar interfaces de usuario de forma rápida con muy pocas líneas de código Swift.

Herramientas de Desarrollo (SDK y Entornos)

Xcode: El entorno de desarrollo integrado (IDE) oficial y obligatorio de Apple. Solo funciona en ordenadores macOS. Incluye el editor de código, herramientas de diseño gráfico, depuradores y los simuladores de iPhone.

iOS SDK: El kit de desarrollo nativo que incluye las APIs esenciales de Apple, como *Cocoa Touch* (para la interfaz táctil), *Core Animation* y marcos de realidad aumentada.

Swift Package Manager (SPM): La herramienta oficial integrada en Xcode para gestionar, descargar e instalar librerías de código creadas por terceros.

TestFlight: Plataforma oficial de Apple utilizada por los desarrolladores para distribuir versiones beta (de prueba) de sus aplicaciones a usuarios externos antes de publicarlas.

PLATAFORMAS DE DESARROLLO

Android

Android Studio es el Entorno de Desarrollo Integrado (IDE) oficial para el sistema operativo Android. Está construido sobre el software *IntelliJ IDEA* de la empresa JetBrains y fue presentado por Google en 2013 para sustituir al antiguo sistema Eclipse. Es una plataforma "todo en uno" que proporciona a los programadores absolutamente todas las herramientas necesarias para escribir código, diseñar interfaces visuales, detectar errores y compilar aplicaciones .

Sistema de Construcción Flexible Gradle como método de compilación.

Emulador de Android Integrado.

Editor de Diseño Visual Avanzado

Diseño declarativo y visual.

Analizadores de Rendimiento (Profilers).

Asistentes de Inteligencia Artificial (Gemini en Android Studio)

Directrices de UI UX: Material Design (evolucionada actualmente a Material You). Concepto visual: Se basa en el comportamiento de los materiales en el mundo real (papel y tinta). Utiliza sombras digitales, capas superpuestas y animaciones fluidas para dar una sensación de profundidad física. Personalización dinámica: Con *Material You*, el sistema extrae automáticamente los colores del fondo de pantalla elegido por el usuario y los aplica a los botones, menús e interfaces de todas las aplicaciones compatibles. Componentes típicos: Uso de menús laterales ocultos (Navigation Drawer), botones flotantes de acción rápida en la esquina inferior (Floating Action Button) y pestañas superiores.

Tienda de apps Google Play Store :

Es la plataforma oficial de distribución de aplicaciones para Android, operada por Google y lanzada originalmente en 2008 bajo el nombre de *Android Market*.

Modelo abierto y libertad: Aunque es la tienda principal, Android permite la existencia y el uso de tiendas de terceros de forma legal y nativa (como la *Galaxy Store* de Samsung o la *Appstore* de Amazon). También permite el *sideloading*, que consiste en descargar e instalar archivos .apk directamente desde cualquier navegador web.

Proceso de publicación ágil: Google utiliza algoritmos automatizados basados en inteligencia artificial para revisar las aplicaciones que envían los desarrolladores. Debido a esto, una aplicación nueva suele ser aprobada y subirse a la tienda en horas o días

Estadísticas de mercado: Cuenta con el mayor volumen de aplicaciones del mundo (más de 2.5 millones de apps disponibles) debido a la inmensa cantidad de dispositivos Android globales, aunque el gasto promedio por usuario en descargas es menor que en su competidor.

IOS

Xcode es el Entorno de Desarrollo Integrado (IDE) oficial y propietario creado por Apple Inc. Fue lanzado originalmente en 2003 y ha evolucionado hasta convertirse en una herramienta centralizada "todo en uno". A diferencia de la plataforma de Android, Xcode tiene una restricción de hardware absoluta: solo se puede instalar y ejecutar en ordenadores Mac con el sistema operativo macOS. Es el único canal de software capaz de compilar el código nativo para los procesadores de Apple y el único autorizado para empaquetar y subir aplicaciones a la App Store.

SwiftUI y Previsualizaciones Dinámicas (Previews)

Diseño declarativo: Integra de forma nativa el framework SwiftUI, permitiendo construir interfaces de usuario modernas utilizando muy pocas líneas de código.

Optimización extrema: Xcode incluye una aplicación independiente muy potente llamada Instruments.

Fugas de memoria (Leaks): Rastrea el código en tiempo real para encontrar bloqueos o consumo fantasma de memoria RAM que puedan ralentizar el iPhone o agotar su batería.

Distribución interna: Xcode está conectado directamente con los servidores de Apple. Permite enviar versiones de prueba de la app a TestFlight con un solo clic para que usuarios externos (betatesters) la prueben antes de su lanzamiento.

Firma de código y seguridad: Gestiona de forma automática los certificados de seguridad digitales y los perfiles de aprovisionamiento necesarios para que el software sea reconocido como seguro y legal por el sistema iOS.

Directrices UI UX: Human Interface Guidelines (Guías de Interfaz Humana), priorizando la claridad, la deferencia al contenido y la profundidad. Minimalismo extremo, la tipografía limpia, los contrastes marcados y el uso de la transparencias. Consistencia estricta: mismos patrones para todas las apps. Componentes típicos: Barra de navegación fija en la parte inferior de la pantalla (Tab Bar), botones de retroceso siempre en la esquina superior izquierda y el uso de gestos físicos (como deslizar el dedo desde el borde izquierdo para volver atrás).

Apple App Store (iOS)

Es la tienda de aplicaciones exclusiva para los dispositivos de Apple, presentada en 2008 junto con el lanzamiento del iPhone 3G.

Monopolio y control estricto: Históricamente ha sido el único canal autorizado por Apple para instalar software en un iPhone. Aunque las normativas internacionales (como la Ley de Mercados Digitales de la Unión Europea) han obligado a Apple a permitir tiendas alternativas en ciertas regiones, el sistema sigue blindado para que el usuario común solo use la App Store oficial, manteniendo un ecosistema muy controlado.

Revisión humana rigurosa: Cada aplicación o actualización enviada por un desarrollador pasa por un filtro manual y exhaustivo realizado por ingenieros de Apple. Evalúan que la app no tenga errores (bugs), que sea segura, que respete estrictamente la privacidad del usuario y que cumpla al pie de la letra con las Human Interface Guidelines. El proceso puede tardar varios días y los rechazos son muy comunes.

Rentabilidad económica: Aunque tiene un catálogo de aplicaciones ligeramente menor que Google Play (alrededor de 1.8 millones), la App Store es drásticamente más rentable. Los usuarios de iOS estadísticamente gastan mucho más dinero en compras dentro de las aplicaciones y suscripciones premium, lo que atrae prioritariamente a los desarrolladores de software

VENTAJAS Y DESVENTAJAS

Desarrollar para Android

Ventajas

Barrera de entrada económica: La licencia de desarrollador de Google cuesta solo 25 \$ (pago único de por vida), lo que facilita empezar con presupuestos bajos.

Libertad de hardware: Puedes programar y compilar tus aplicaciones usando cualquier ordenador (Windows, Linux o Mac) y probarlas en cualquier teléfono.

Flexibilidad en las directrices: Google es permisivo con el diseño visual, lo que te da total libertad creativa para experimentar con la interfaz de usuario.

Publicación casi inmediata: Las aplicaciones se revisan mediante sistemas automáticos rápidos, estando disponibles en la tienda en pocas horas.

Distribución multicanal: No dependes solo de Google Play; puedes distribuir tu app en la tienda de Samsung, Amazon o directamente en tu web corporativa.

Acceso profundo al sistema: Android permite interactuar directamente con funciones internas del teléfono como el almacenamiento compartido o la gestión de archivos.

Comunidad global gigante: Al tener la mayor cuota de mercado del mundo, hay millones de tutoriales, librerías y soluciones en foros de programación.

Desventajas

Fragmentación extrema de dispositivos: Tienes que optimizar tu app para miles de pantallas, procesadores y marcas diferentes, lo que duplica el tiempo de desarrollo.

Múltiples versiones del sistema: Millones de usuarios usan versiones antiguas de Android porque los fabricantes no los actualizan; debes adaptar tu código para que no falle en sistemas viejos.

Menor rentabilidad por usuario: Estadísticamente, los usuarios de Android descargan más aplicaciones gratuitas y son menos propensos a pagar por suscripciones.

Piratería y seguridad: Al permitir la instalación de archivos externos (.apk), es extremadamente fácil que modifiquen o pirateen tu aplicación de forma ilegal.

Entorno de simulación pesado: El emulador oficial de Android Studio consume muchísima memoria RAM y recursos, ralentizando ordenadores de gama media.

Complejidad en Gradle: El sistema de construcción y dependencias de Android suele dar errores de configuración complejos de solucionar tras actualizar el IDE.

Pérdida de rendimiento por ART: Aunque la máquina virtual de Android funciona bien, siempre tendrá un pequeño margen de retraso frente a la ejecución nativa directa.

Desarrollar para iOS (iPhone)

Ventajas

Usuarios más rentables: El público de Apple gasta estadísticamente mucho más dinero en compras dentro de la app, aplicaciones de pago y suscripciones mensuales.

Poca fragmentación: Solo tienes que diseñar e integrar tu aplicación para una lista muy pequeña de dispositivos (los modelos de iPhone activos del mercado).

Simulador ultra veloz: Al no emular hardware completo, el simulador de Xcode arranca de inmediato y funciona con una fluidez idéntica a un teléfono real.

Actualizaciones masivas: Casi todos los usuarios de iPhone actualizan a la última versión de iOS en pocas semanas; puedes usar las últimas tecnologías sin miedo.

Ecosistema Swift moderno: El lenguaje Swift y la herramienta SwiftUI permiten escribir código limpio, seguro y diseñar pantallas estéticas en tiempo récord.

Compilación nativa directa: Al programarse directamente para los potentes procesadores de Apple, las apps consiguen el mejor rendimiento y consumo de batería posible.

Mayor seguridad frente a piratería: El control estricto del sistema hace que sea prácticamente imposible duplicar o piratear tu aplicación fuera de los canales oficiales.

Desventajas

Dependencia obligatoria de Mac: Estás obligado a comprar un ordenador de Apple (macOS) para compilar y subir tus aplicaciones; no existe software oficial para Windows o Linux.

Suscripción anual costosa: La licencia de desarrollador de Apple cuesta 99 \$ *al año*; si dejas de pagarla, eliminan inmediatamente tus aplicaciones de la tienda.

Revisiones humanas estrictas: El comité de revisión de Apple inspecciona tu app manualmente. Pueden rechazar tu trabajo por pequeños fallos visuales o técnicos.

Rigidez en el diseño (HIG): Si tu interfaz se aleja demasiado de los estándares visuales que exige Apple, tu aplicación será rechazada antes de ver la luz.

Ecosistema cerrado y monopolístico: Dependes de las decisiones comerciales de la App Store; si Apple decide cambiar sus normas, tu modelo de negocio puede peligrar.

Pérdida de acceso al sistema: Por motivos de privacidad, iOS bloquea el acceso de las apps a gran parte del hardware y archivos del sistema de forma estricta.

Curva de aprendizaje inicial: Conceptos avanzados obligatorios como la gestión de certificados de seguridad digitales y perfiles de aprovisionamiento son confusos para principiantes.

Tabla Comparativa

	Android	iOS
Plataforma / IDE Oficial	Android Studio (Funciona en Windows, Linux y Mac).	Xcode (Exclusivo y obligatorio para macOS).
Entornos en común (x86)	Permite programar nativamente o usar frameworks como Flutter/React Native en VS Code.	Requiere editores alternativos para el código, pero siempre necesita macOS para compilar.
Lenguaje Moderno Oficial	Kotlin, Java	Swift (Rápido y de sintaxis limpia).
Filosofía de Diseño (UI)	Material Design / Material You: Basado en profundidad física, capas (eje Z) y colores dinámicos adaptados al fondo de pantalla.	Human Interface Guidelines (HIG): Basado en minimalismo extremo, transparencias de fondo y una estética muy limpia.
Naturaleza de las Guías (UI/UX)	Funciona como directrices flexibles. Google te aconseja cómo diseñar, pero no prohíbe publicar si no las sigues.	Funciona como directrices estrictas. Apple revisa visualmente tu app y la rechaza si rompe sus reglas de diseño.
Modelo de Seguridad General	Abierto pero aislado: Cada app corre en su propio Sandbox bajo un usuario único de Linux, impidiendo el espionaje entre apps.	Cerrado y hermético: Aislamiento estricto de archivos del sistema y políticas agresivas de privacidad (anti-rastreo).
Sistema de Construcción	Gradle (Altamente flexible pero propenso a errores de configuración en proyectos grandes).	Swift Package Manager / Xcode Build System (Integrado, automático y optimizado).
Entorno de Pruebas	Emulador: Recrea el hardware completo del teléfono; consume mucha memoria RAM en la computadora.	Simulador: Ejecuta el código adaptado al procesador de la computadora; es ultra rápido y fluido.
Tienda Oficial de Apps	Google Play Store: Revisión automatizada por Inteligencia Artificial; publicación rápida en pocas horas.	Apple App Store: Revisión manual por ingenieros de Apple; exigente con la calidad y tarda varios días.
Distribución Externa	Permite tiendas de terceros (Amazon, Samsung) y descargas libres desde la web del desarrollador.	Altamente restringida y bloqueada por defecto a la tienda oficial de Apple.
Coste de Licencia	25 USD (Pago único de por vida).	99 USD (Suscripción obligatoria que se debe renovar todos los años).
Fragmentación	Muy Alta: Tienes que optimizar el código para miles de pantallas, procesadores y marcas distintas.	Muy Baja: Gama reducida de modelos de iPhone controlados y actualizados en masa por la propia Apple.

Análisis personal

La elección de por donde empezar a desarrollar aplicaciones móviles para Android o IOS en mi caso personal es sencilla, tengo conocimientos de Java C y C++ y quisiera ampliar esto para el desarrollo de aplicaciones móviles que permitan interactuar con dispositivos LOT , como va a ser un banco de pruebas, y mis equipos son x86 y Android y el primer público general sería Android, posteriormente si mis proyectos requieren también cuota de mercado iOS adelante con ello. Por otra parte la inversión de equipos Apple es más costosa.

En el futuro es posible que se utilice más Flutter para escribir código una sola vez pero para nicho de mercado LOT Java con Android se adapta mucho mejor. Otro punto es la cantidad de librerías ya disponibles para el mundo de LOT y la facilidad de interactuar con dispositivos ESP32 usando Android.